ENGE707 · WEEK 4

Machine Learning Techniques

From a hard-coded rule to a model that learns its own rule — supervised, unsupervised, and reinforcement learning, and the linear models underneath them, built step by step in code.

LECTURE ROADMAP

What you'll be able to do by the end of today

- Place machine learning inside the history of AI — from hand-written rules to models that learn from data
- Explain the difference between supervised, unsupervised, and reinforcement learning
- Write, line by line, the full scikit-learn workflow: prepare features, split, fit, predict, evaluate
- Train and interpret a Linear Regression model, including its coefficients and R^2
- Train and interpret a Logistic Regression model, including predicted probabilities, accuracy, and a confusion matrix
- Repeat K-means clustering on a real, larger dataset and interpret the result against a true label

BUILDING ON WHAT YOU KNOW

Lecture 1 wrote one hard-coded rule with np.where(). Lecture 2–3 cleaned and explored a real patient dataset (df_etl_clean).

Today, that same cleaned data is handed to a model that works out its own rule from examples — and we build that workflow one code cell at a time, exactly like the worked labs.

PART I

Where machine learning fits in the history of AI

A little context before the code.

BEFORE WE CODE

Historical paradigms of AI

1950s–80s	**Rule-based / symbolic AI**	Humans write every rule by hand (“if temperature > 37.7 and pH > 7.65, flag deteriorating”). Powerful for narrow, well-understood problems, brittle everywhere else.
1980s–90s	**Expert systems**	Rules are collected from human experts into large knowledge bases. Still hand-built — the bottleneck becomes writing enough rules to cover the real world.
1990s–today	**Machine learning**	Instead of writing the rule, we show the system many labelled examples and let it find the rule itself. This is where supervised, unsupervised, and reinforcement learning all sit.
2010s–today	**Deep learning & generative AI**	Machine learning at large scale using many-layered neural networks — powers image recognition, large language models, and today's generative AI tools.

Today's lecture lives in the “machine learning” row — specifically, the simplest and most interpretable family of models inside it: linear models.

THE CORE IDEA

What does “learning” actually mean here?

HAND-WRITTEN RULE (Lecture 1)

```
df["prediction"] = np.where(
    (df["temperature"] > 37.7)
    & (df["pH"] > 7.65),
    "deteriorating", "stable"
)
```

A human decided 37.7 and 7.65 were the thresholds that mattered — by eye, in advance.

LEARNED RULE (today)

```
model = LogisticRegression()
model.fit(X_train, y_train)

# model has now chosen its own
# weights and decision threshold
```

The model looks at hundreds of labelled examples and works out its own weights — no human had to guess the right numbers.

THE THREE FAMILIES

Three ways a machine can learn

Supervised learning

Learns from labelled examples — every input already has a known correct answer.

e.g. is this patient's diagnosis arrhythmia or not?

Unsupervised learning

Learns from unlabelled examples — finds structure or groups on its own.

e.g. do natural patient sub-groups exist in the ECG data?

Reinforcement learning

Learns by acting and receiving a reward or penalty over time — no fixed correct-answer set at all.

e.g. which treatment sequence maximises patient outcomes?

PART II

Supervised learning

Learning from examples that already have the right answer.

LEARNING FROM LABELLED EXAMPLES

What makes learning “supervised”?

- Every training row has a known, correct answer attached — the label
- The model is shown many (features, label) pairs and adjusts itself to get as many right as possible
- Once trained, it predicts a label for new, unseen rows
- Two flavours: classification (label is a category) and regression (label is a number)
- “Supervised” refers to the labels acting like a teacher checking the model's answers during training

ACROSS OTHER DOMAINS

Banking: past transactions labelled fraud / not fraud → a fraud model

Retail: past customers labelled churned / retained → a churn model

Manufacturing: past sensor readings labelled failed / ok → a failure model

Today's example: past patients labelled arrhythmia / normal → a diagnosis-support model

TWO FLAVOURS OF SUPERVISED LEARNING

Classification vs regression, side by side

CLASSIFICATION

Predicts a category

“Is this patient's diagnosis arrhythmia or normal?”

Output: one of a fixed set of labels (arrhythmia_present = True / False).

Model used today: Logistic Regression — a linear model adapted to output a category.

REGRESSION

Predicts a number

“What is this patient's resting heart rate likely to be?”

Output: any numeric value, e.g. 78.4 bpm.

Model used today: Linear Regression — the same linear idea, left as a raw number.

CONCEPT BRIDGE

Feature, label and prediction — the table view

Supervised learning starts with examples that already have the right answer.

Patient	QRS	Heart rate	Age	Label
1	82	70	35	Normal
2	110	93	72	Arrhythmia
3	95	65	48	Normal
4	122	104	80	Arrhythmia
5	88	75	56	Normal

features
what the model can see

label
the answer during training

New patient
QRS = 116
HR = 97
Age = 70

Model predicts
Arrhythmia?

Layman idea:
During training, the answer column teaches the model.
Later, a new patient has features only; the model estimates the missing answer.

CONCEPT BRIDGE

Learning means choosing useful weights

The model is still just a formula — the difference is that it learns the numbers from data.

Three patient measurements — **Learned weights** — **Raw score**

Example patient

QRS duration = 110
Heart rate = 90
Age = 70

Toy equation

score = -7
+ 0.04×QRS
+ 0.03×HR
+ 0.02×Age

Calculation

-7 + 4.4 + 2.7 + 1.4
= 1.5

Higher score
= more arrhythmia-like

The actual model learns its own weights; these numbers are only for understanding the idea.

PICKING UP THE SAME SESSION

Continuing from the last cell of Lecture 2–3

```
# ...the last cell of the Lecture 2–3 notebook:
table = pd.crosstab(
    df_etl_clean["sex"], df_etl_clean["arrhythmia_present"]
)
chi2, p, dof, expected = stats.chi2_contingency(table)
print(f"chi2 = {chi2:.2f}, p = {p:.6f}")

# Lecture 4 starts right here — same kernel,
# same df_etl_clean, nothing reloaded
print(df_etl_clean.shape)
```

WHY NOTHING IS RELOADED

df_etl_clean, sex, and arrhythmia_present are still sitting in memory exactly as the chi-square test above left them — this lecture opens the same notebook session, not a fresh one.

LINE BY LINE

1. **`table = pd.crosstab(...)`**
   Recap from Lecture 2–3: cross-tabulates sex against arrhythmia_present into a small counts table.
2. **`chi2, p, dof, expected = stats.chi2_contingency(table)`**
   Recap: runs the chi-square test on that table, unpacking four results at once.
3. **`print(f"chi2 = ...")`**
   Recap: prints the test statistic and p-value — this is where Lecture 2–3 ended.
4. **`print(df_etl_clean.shape)`**
   New this lecture: confirms df_etl_clean (448 rows) is still sitting in memory, unchanged, before any model gets built.

MODEL 1 OF 2

Linear regression: fitting the best straight line

- A linear model predicts an output as a weighted sum of the inputs, plus a baseline value
- For one feature: predicted value = slope × feature + intercept — the equation of a straight line
- “Fitting” means searching for the slope and intercept that make the line's predictions as close as possible to the real, observed values
- With several features, each one gets its own weight, and the model adds them all together
- We will build this in seven small steps: check correlations → prepare data → split → fit → inspect weights → predict → evaluate — the same shape every supervised model in this course follows

WHY START HERE

Linear models are the simplest useful models in machine learning — easy to train, fast to run, and easy to explain to a non-technical audience.

They are the standard first baseline: before reaching for something more complex, check whether a straight line already does a reasonable job.

BEFORE THE CODE

How linear regression is calculated, by hand

- Linear regression finds the best-fitting straight line through a set of data points
- The equation of that line: predicted value = slope × feature + intercept

$$\hat{y} = m \cdot x + b$$

WHAT EACH SYMBOL MEANS

x is the input feature, ŷ (“y-hat”) is the predicted output.

m is the slope of the line — how much ŷ changes per unit of x.

b is the intercept — the value of ŷ when x is zero.

WORKED EXAMPLE

Suppose temperature predicts wound-healing days:

Temperature (x)	Healing days (y)
30	5
32	7
34	9

In real datasets the points rarely form a perfect line — regression searches for the m and b that fit them as closely as possible, on average.

BEFORE THE CODE

Calculating the slope and intercept, by hand

Slope — how much y changes per unit of x

$$m = (y_2 - y_1) / (x_2 - x_1)$$

$$m = (9-5)/(34-30) = 1$$

Every 1°C increase in temperature → healing time increases by 1 day.

Intercept — the baseline value when x = 0

$$b = y - m \cdot x$$

$$b = 5 - (1 \times 30) = -25$$

Fitted equation: $\hat{y} = x - 25$

For a temperature of 33°C: $\hat{y} = 33 - 25 = 8$ days

WHY “LEAST SQUARES”?

The gap between an actual value and its prediction is called an error. Linear regression picks the line that minimises the sum of squared errors:

$$\Sigma (y_i - \hat{y}_i)^2$$

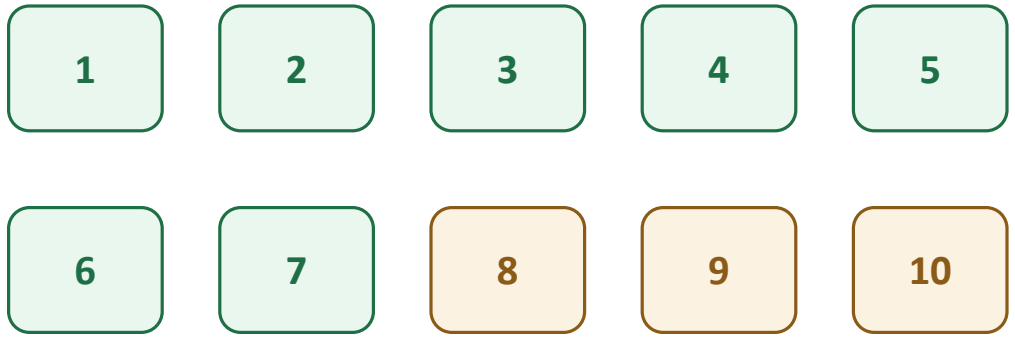
Squaring stops positive and negative errors from cancelling out, and punishes large errors more than small ones. In plain terms: the best line is the one with the smallest total squared distance to every point.

CONCEPT BRIDGE

Train/test split = practice questions vs exam questions

The model must be tested on patients it did not learn from.

10 small patient cards



Training set
70%: model learns here

Test set
30%: model is graded here

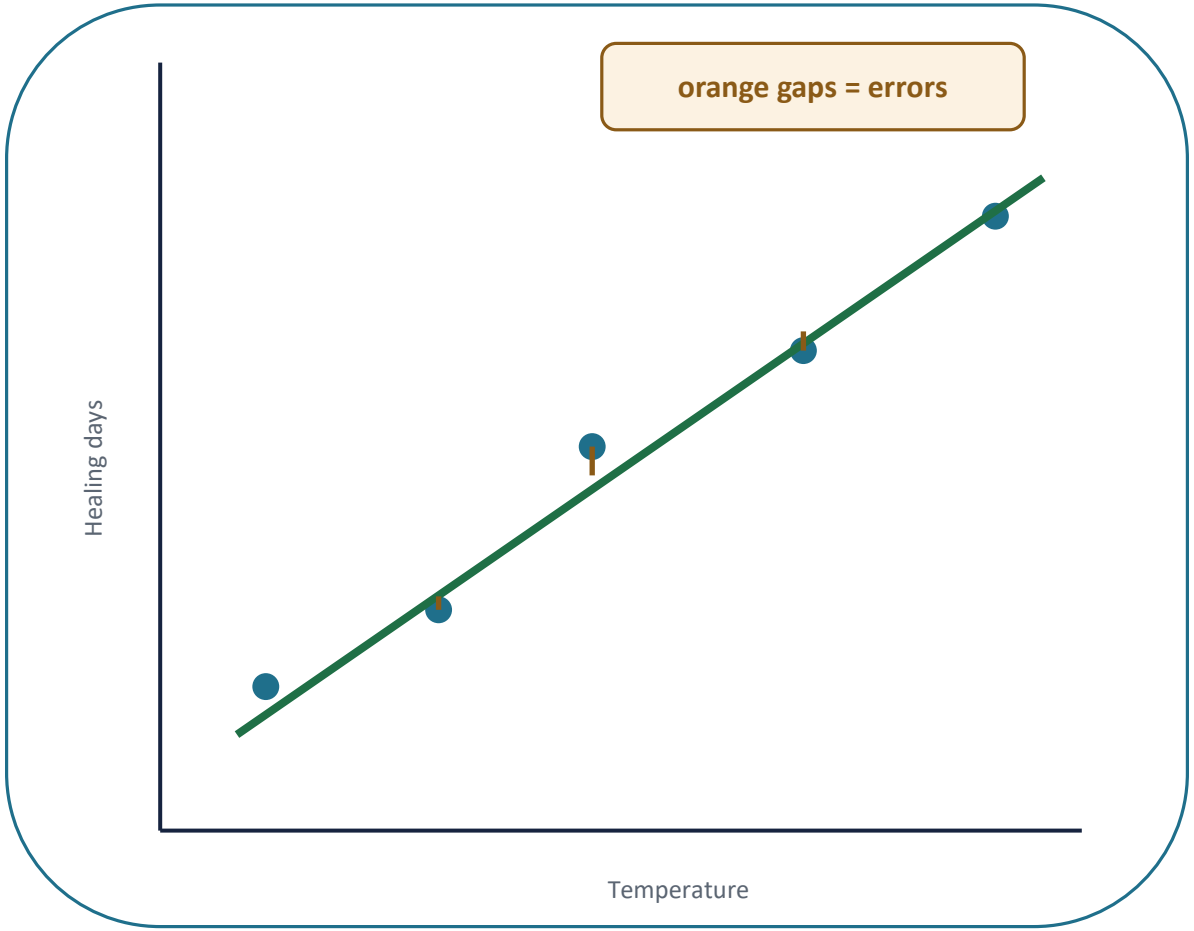
Why it matters:
If the model sees the same rows during training and testing, it may only be memorising.
A held-out test set checks whether it generalises to a new patient.

In the real dataset: 448 patients → about 313 train and 135 test.

CONCEPT BRIDGE

Linear regression: choosing the line with the smallest misses

The line is judged by how far its predictions are from the real values.



x	actual y	line predicts	error
30	5	5.2	-0.2
32	7	6.8	+0.2
34	9	8.1	+0.9
36	10	10.4	-0.4
38	12	11.9	+0.1

The best line is not the one that perfectly hits one point. It is the one that is collectively closest to all points.

CONCEPT BRIDGE

Why “squared” errors?

Squaring stops plus and minus errors cancelling out and makes big misses hurt more.

Patient	Actual	Predicted	Error	Error²
1	70	72	-2	4
2	82	78	+4	16
3	59	62	-3	9
4	90	83	+7	49
5	74	75	-1	1

Total squared error
4 + 16 + 9 + 49 + 1 = 79

Regression tries many possible lines.
It keeps the line with the smallest total squared error.

Layman shortcut: the best line is the one with the smallest total “ouch” from its misses.

UNDERSTANDING THE CHECK

What is correlation, really?

Correlation measures how strongly two numbers move together, on a scale from −1 to +1. Before picking a feature for a model, we check its correlation with the target instead of guessing which one “sounds” relevant.

SIMPLE EXAMPLE

Two lists that rise together, step for step — e.g. temperature and healing days from the last slide — have a correlation near +1.

Two lists where one rises as the other falls — e.g. QT interval and heart_rate — have a correlation near −1.

Two lists with no real relationship — e.g. a patient's shoe size and their heart_rate — have a correlation near 0.

In one sentence: *The sign tells you the direction of a relationship; the distance from zero — not the sign — tells you how strong it actually is.*

HOW TO INTERPRET A CORRELATION

+1.0

perfect positive relationship

+0.6

a strong positive relationship

0.0

no linear relationship at all

−0.6

a strong negative relationship — just as useful as +0.6

−1.0

perfect negative relationship

STEP 1 OF 7

Check which features actually correlate with heart_rate

```
clinical_cols = ["age", "sex", "height", "weight", "qrs_duration",
    "pr_interval", "qt_interval", "t_interval", "p_interval",
    "qrs_angle", "t_angle", "p_angle", "qrst_angle", "bmi"]

correlations = df_etl_clean[clinical_cols +
["heart_rate"]].corr()["heart_rate"]
print(correlations.abs().sort_values(ascending=False))
```

LINE BY LINE

1. **`clinical_cols = [...]`**
   Restricts the check to the 14 named clinical columns, not all ~280 raw columns.

2. **`correlations = df_etl_clean[...].corr()["heart_rate"]`**
   Computes correlation of every clinical column against heart_rate.

3. **`print(correlations.abs().sort_values(ascending=False))`**
   .abs() ranks by strength, not sign — otherwise negatives sort last.

qt_interval is the only relationship worth calling strong — $r^2 \approx 0.03$ for the next-best column means it's barely above noise with 14 columns tested. One honest feature beats two where the second is fabricated.

REAL OUTPUT (from the course notebook)

qt_interval	**0.617**
height	0.177
p_angle	0.126
age	0.122
weight	0.082
sex	0.080
t_angle	0.074
p_interval	0.063
qrs_duration	0.005

values shown are |correlation| · green = the one feature strong enough to justify using

STEP 2 OF 7

Import the tools and choose the features

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

# df_etl_clean is still the same object from Lecture 2–3
X = df_etl_clean[["qt_interval"]]
y = df_etl_clean["heart_rate"]

print("X:", X.shape, " y:", y.shape)
# X: (448, 1)  y: (448,)   <- illustrative output
```

WHAT EACH LINE DOES

The three import lines pull in the LinearRegression model class, the train/test splitting tool, and a metric (R^2) used later to score the model — nothing runs yet, they just load the tools.

X is the table of inputs (features) the model is allowed to look at, still the same df_etl_clean sitting in memory from last lecture — qt_interval alone, the only column Step 1 found a genuinely strong correlation for.

y is the single column the model is trying to predict — here, heart_rate. The print confirms both have 448 rows before anything gets trained.

LINE BY LINE

1. **`from sklearn.linear_model import LinearRegression`**
   Loads the model class — the object that will hold the learned slope/intercept once trained.
2. **`from sklearn.model_selection import train_test_split`**
   Loads the tool used in Step 3 to hold back a test set.
3. **`from sklearn.metrics import r2_score`**
   Loads the scoring function used in Step 7.
4. **`X = df_etl_clean[["qt_interval"]]`**
   Builds a one-column feature table — double brackets keep it a DataFrame, which scikit-learn requires even for a single feature.
5. **`y = df_etl_clean["heart_rate"]`**
   Pulls out the single target column — single brackets give a Series, exactly what y needs to be.
6. **`print("X:", X.shape, " y:", y.shape)`**
   Confirms both have 448 rows before anything gets trained.

STEP 3 OF 7

Split the data into training and test sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("Train:", X_train.shape, " Test:", X_test.shape)
# Train: (313, 1)  Test: (135, 1)   <- illustrative
```

WHY SPLIT AT ALL

If the model is trained and then tested on the same rows, it can score well simply by memorising — that tells you nothing about how it will do on a new patient.

test_size=0.3 holds back 30% of the rows purely for testing; the model never sees them during .fit().

The function returns four pieces in a fixed order — X_train, X_test, y_train, y_test — always in that order, which is easy to get wrong the first few times.

LINE BY LINE

1. **`X_train, X_test, y_train, y_test = train_test_split(`**
   Unpacks four results at once, always in this exact order.

2. **`X, y, test_size=0.3, random_state=42`**
   Splits X and y together, holding back 30% for testing, with a fixed random shuffle.

3. **`)`**
   Closes the function call — the split happens the moment this line runs.

4. **`print("Train:", X_train.shape, " Test:", X_test.shape)`**
   Confirms the 70/30 split actually happened — 313 training rows, 135 test rows.

UNDERSTANDING EVALUATION

Why do we split the data, really?

A model that is trained and tested on the exact same rows can score perfectly just by memorising the answers — that tells you nothing about how it will perform on a new patient it has never seen.

SIMPLE EXAMPLE

Imagine a student who only ever practises the exact questions that will be on the exam — they can score 100% without having learned the underlying topic at all.

A fairer test uses different questions from the same topic — that's what the held-out 30% test set does for a model.

test_size=0.3 sets aside 135 of the 448 patients that the model never sees during .fit() — they exist purely to check generalisation afterwards.

In one sentence: *Splitting the data is what turns a score from "how well did it memorise" into "how well will it generalise to a new patient."*

WHAT THE SPLIT GIVES YOU

`Train (70%)`

what the model is allowed to learn from

`Test (30%)`

what the model is graded on, afterwards

`random_state=42`

makes the split reproducible, not random each run

CONCEPT BRIDGE

R^2: did the model beat a very basic guess?

R^2 compares the regression model against "just predict the average for everyone."

Actual heart rates
60, 70, 75, 85, 95

No-model guess
Average = 77 for everyone

Linear model
uses QT interval to make different guesses

R^2 answers one question:

"How much of the variation in heart_rate did the model explain compared with just predicting the average?"

R^2 = 0.37 means the model explains about 37% of the differences between patients.
The rest is still unexplained by this one feature.

R^2 is for regression, not classification.

STEP 4 OF 7

Fit the model — this is the “learning” step

```
model = LinearRegression()
model.fit(X_train, y_train)

print("Model trained on", len(X_train), "patients")
```

LINE BY LINE

1. **`model = LinearRegression()`**
   Creates an empty model object — no weights yet, just default settings.

2. **`model.fit(X_train, y_train)`**
   The actual learning step: searches for the slope of each feature and one intercept that best fit y_train, using ordinary least squares under the hood.

3. **`print("Model trained on", len(X_train), "patients")`**
   Confirms the fit ran, and on how many rows — 313, matching Step 3's training split.

This one line — model.fit(X_train, y_train) — is the entire “learning” in machine learning. Everything before it is preparation; everything after it is using what was learned.

STEP 5 OF 7

Inspect what the model actually learned

```
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Coefficients: [-0.58]
# Intercept: 297.5   <- illustrative output
```

LINE BY LINE

1. **`print("Coefficients:", model.coef_)`**
   Prints one weight per feature — with a single feature, this is a one-item array, [qt_interval_weight].

2. **`print("Intercept:", model.intercept_)`**
   Prints the baseline prediction when qt_interval is zero — not physically meaningful on its own, but completes the equation.

THE FULL EQUATION

```
heart_rate ≈
297.5 − 0.58 × qt_interval
```

NUMBERS WILL VARY

The coefficient is negative — matching Step 1's real correlation (−0.617): a longer QT interval is associated with a lower heart_rate. A coefficient of −0.58 means each extra millisecond of QT interval is associated with about 0.58 fewer bpm.

The exact coefficient depends on your specific train/test split and the real data — treat the values above as illustrative, not a fixed answer key. Run the lab's code on your own cleaned dataset to see your own numbers.

STEP 6 OF 7

Predict on data the model has never seen

```
predictions = model.predict(X_test)

comparison = pd.DataFrame({
    "actual": y_test,
    "predicted": predictions
})
print(comparison.head())
```

LINE BY LINE

1. **`predictions = model.predict(X_test)`**
   Applies the learned equation from Step 5 to every row of X_test in one go — no loop needed.

2. **`comparison = pd.DataFrame({"actual": y_test, "predicted": predictions})`**

   Builds a small two-column table so actual and predicted values sit side by side.

3. **`print(comparison.head())`**
   Shows the first 5 rows — a sanity check before trusting any single summary score.

ILLUSTRATIVE OUTPUT

actual	predicted
68	70.1
82	79.4
59	61.8
90	83.5
74	75.2

STEP 7 OF 7

Score the model with R²

```
r2 = r2_score(y_test, predictions)
print("R²:", round(r2, 2))
# R²: 0.37   <- illustrative output
```

LINE BY LINE

1. **`r2 = r2_score(y_test, predictions)`**
   Compares predicted values against the real y_test values and computes R^2 — how much of the variation the model explains, from 0 to 1.

2. **`print("R²:", round(r2, 2))`**
   Prints the score rounded to 2 decimals — 0.37 lines up with qt_interval's correlation of −0.617 ($0.617^2 \approx 0.38$), so this number isn't a surprise, it's the same relationship measured a different way.

THE SEVEN STEPS, RECAPPED

1. Check correlations with heart_rate
2. Prepare X and y
3. Split into train / test
4. model.fit(X_train, y_train)
5. Inspect coef_ and intercept_
6. model.predict(X_test)
7. Score with r2_score()

Every supervised model in this course follows this same shape — logistic regression, next, skips straight to Step 2 since its three features are already justified.

UNDERSTANDING THE SCORE

What is R^2, really?

R^2 ("R-squared") summarises how well the model's predictions explain the differences in heart_rate among patients — it's calculated in Step 7, from the same predictions vs actual comparison you just printed in Step 6.

SIMPLE EXAMPLE

Imagine five patients with heart rates: 60, 70, 75, 85, 95 bpm.

A very basic guess: predict the average (77 bpm) for everyone — that's the "no model" baseline.

Linear regression instead uses each patient's qt_interval. R^2 tells us how much better that is than just guessing the average for every patient.

In one sentence: *R^2 tells us what percentage of the differences in the output can be explained by the features given to the model.*

HOW TO INTERPRET R^2

$R^2 = 1.00$

predictions perfectly match the actual values

$R^2 = 0.70$

the model explains 70% of the variation

$R^2 = 0.37$

today's result — the model explains 37%

$R^2 = 0$

no better than predicting the average

$R^2 < 0$

possible on unseen test data — worse than the average

MODEL 2 OF 2

Logistic regression: a linear model for categories

- Starts from the same idea as linear regression — a weighted sum of the features
- But that raw sum is squeezed into a probability between 0 and 1 using an S-shaped curve (the sigmoid function)
- A threshold (usually 0.5) turns that probability into a category: above it → one class, below it → the other
- The model still learns a weight per feature — it is a linear model wearing a classification hat
- Despite the name, logistic regression is a classification method, not a regression one
- We will build it in six steps too — the same shape as linear regression, with two extra stops: probabilities, and a confusion matrix

IN TODAY'S EXAMPLE

Predicting arrhythmia_present from qrs_duration, heart_rate, and age.

The model doesn't just say “yes/no” — it can also return the underlying probability, e.g. “0.82 chance of arrhythmia”, which is often more useful clinically than a bare label.

CONCEPT BRIDGE

Where did the sigmoid come from?

Logistic regression needs to turn any raw score into a probability between 0 and 1.

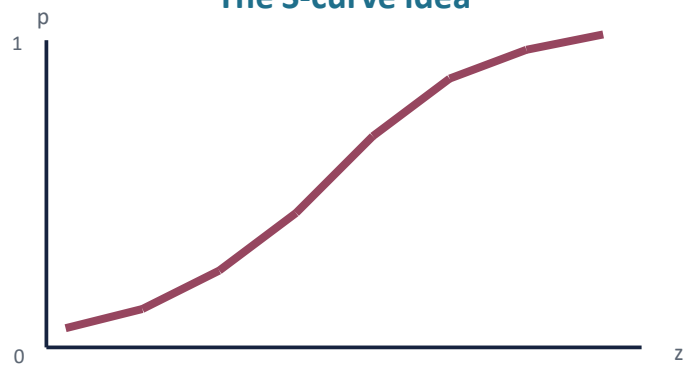
Raw score z
can be negative, zero, or positive

Sigmoid
“squashes” the score

Probability
always between 0 and 1

Raw score z	Probability p	Plain meaning
-4	0.02	very unlikely
-1	0.27	leans normal
0	0.50	coin flip
+1	0.73	leans arrhythmia
+4	0.98	very likely

The S-curve idea



The raw score is calculated by multiplying each feature by its learned weight and then adding the intercept:

$$z = b + w_1x_1 + w_2x_2 + w_3x_3$$

$$p = \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$$

STEP 1 OF 6

Prepare the features and the label

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# still the same df_etl_clean, same session
X = df_etl_clean[["qrs_duration", "heart_rate", "age"]]
y = df_etl_clean["arrhythmia_present"]

print(X.shape)
print(y.value_counts())
```

LINE BY LINE

1. **`from sklearn.linear_model import LogisticRegression`**
   Loads the classifier class.
2. **`from sklearn.metrics import accuracy_score, confusion_matrix`**
   Loads two new evaluation tools used from Step 4 onward.
3. **`X = df_etl_clean[["qrs_duration", "heart_rate", "age"]]`**
   Builds the feature table — all three justified on the right.
4. **`y = df_etl_clean["arrhythmia_present"]`**
   The yes/no target column clean_data() created back in Lecture 2–3.
5. **`print(X.shape); print(y.value_counts())`**
   Confirms row count and shows straight away how imbalanced the two classes are.

WHY THESE THREE FEATURES

`qrs_duration`	Timing of the heartbeat's main electrical signal — directly relevant to rhythm abnormalities.
`heart_rate`	An abnormally fast or slow rate is itself a sign of arrhythmia.
`age`	Arrhythmia risk generally changes with age — a useful supporting feature.

STEP 2 OF 6

Split the data — keeping both classes balanced

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("Train arrhythmia rate:", round(y_train.mean(), 2))
print("Test arrhythmia rate:", round(y_test.mean(), 2))
```

WHY stratify=y THIS TIME

Lecture 2–3 found that arrhythmia diagnosis is not evenly split across this dataset, and is associated with sex.

Without stratify=y, a random split could by chance put too few (or too many) arrhythmia cases in the test set, making the accuracy score below unreliable.

stratify=y forces both printed rates to land close together — a small addition with a real effect on trustworthy results.

LINE BY LINE

1. **`X_train, X_test, y_train, y_test = train_test_split(`**
   Same four-way unpack as linear regression's split.

2. **`X, y, test_size=0.3, random_state=42, stratify=y`**
   Adds stratify=y this time — keeps the arrhythmia/normal ratio consistent in both halves.

3. **`)`**
   The split runs the moment this line executes.

4. **`print("Train arrhythmia rate:", round(y_train.mean(), 2))`**
   Prints the fraction of arrhythmia cases in the training set.

5. **`print("Test arrhythmia rate:", round(y_test.mean(), 2))`**
   Prints the same for the test set — stratify=y should make these two numbers land close together.

STEP 3 OF 6

Fit the model, then look at probabilities

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

probabilities = model.predict_proba(X_test)
print(probabilities[:5])   # first 5 rows, illustrative
```

LINE BY LINE

1. **`model = LogisticRegression(max_iter=1000)`**
   Creates the classifier; max_iter=1000 gives its optimiser enough attempts to converge on noisier real clinical data.
2. **`model.fit(X_train, y_train)`**
   The learning step — searches for the weight per feature that best separates the two classes.
3. **`probabilities = model.predict_proba(X_test)`**
   Returns two probabilities per row (normal, arrhythmia), always summing to 1.0 — the sigmoid curve made concrete.
4. **`print(probabilities[:5])`**
   Shows the first 5 rows so you can see actual confidence values, not just a bare label.

ILLUSTRATIVE OUTPUT

normal	arrhythmia
0.71	0.29
0.18	0.82
0.55	0.45
0.09	0.91
0.63	0.37

CONCEPT BRIDGE

Five patients: from measurements → score → probability

These toy numbers show the idea; the real model learns its own weights.

Patient	QRS	Heart rate	Age	Score z	p(arrhythmia)	Prediction
A	80	60	30	-1.4	0.20	Normal
B	90	70	50	-0.3	0.43	Normal
C	100	80	60	0.6	0.65	Arrhythmia
D	110	90	70	1.5	0.82	Arrhythmia
E	120	100	80	2.4	0.92	Arrhythmia

Toy score: z = -7 + 0.04×QRS + 0.03×heart_rate + 0.02×age

Sigmoid converts z into p. Then p ≥ 0.50 predicts arrhythmia.

CONCEPT BRIDGE

Probability becomes a decision using a threshold

By default, logistic regression uses 0.50 as the cut-off.

Patient	p(arrhythmia)	Model prediction	Actual diagnosis	Correct?
A	0.20	Normal	Normal	Yes
B	0.43	Normal	Arrhythmia	No
C	0.65	Arrhythmia	Arrhythmia	Yes
D	0.82	Arrhythmia	Arrhythmia	Yes
E	0.92	Arrhythmia	Normal	No

If p < 0.50
Predict Normal

If p ≥ 0.50
Predict Arrhythmia

Accuracy = correct / total = 3 / 5 = 60%

This slide uses five rows only to explain the calculation. In the real code, accuracy uses all test patients.

UNDERSTANDING PROBABILITIES

What is the sigmoid function, really?

Underneath, logistic regression still computes a weighted sum of the features — exactly like linear regression. The sigmoid function is what squeezes that raw, unbounded number into a probability between 0 and 1.

SIMPLE EXAMPLE

A raw weighted sum of −4 becomes a probability near 0.02 (very unlikely).

A raw weighted sum of 0 becomes exactly 0.50 (a coin flip).

A raw weighted sum of +4 becomes a probability near 0.98 (very likely).

The S-shaped curve means most of the movement happens near the middle — far from zero, the probability barely changes.

In one sentence: *Sigmoid turns "how strongly do the features point one way" into a number you can read as a probability and threshold into a decision.*

HOW TO INTERPRET THE PROBABILITY

`0.00 – 0.30`

model leans strongly toward "normal"

`0.30 – 0.70`

model is genuinely uncertain

`0.70 – 1.00`

model leans strongly toward "arrhythmia"

`= 0.50`

the default decision threshold used by .predict()

STEP 4 OF 6

Turn probabilities into predictions, then score

```
predictions = model.predict(X_test)

accuracy = accuracy_score(y_test, predictions)
print("Accuracy:", round(accuracy, 2))
# Accuracy: 0.72   <- illustrative output
```

LINE BY LINE

1. **`predictions = model.predict(X_test)`**
   Runs the same probability calculation as Step 3, then applies the 0.5 threshold to return a clean True/False label.

2. **`accuracy = accuracy_score(y_test, predictions)`**
   Compares predicted labels against the real y_test values and computes the fraction that matched.

3. **`print("Accuracy:", round(accuracy, 2))`**
   Prints the score — 0.72 here means about 72% of held-out patients were classified correctly.

TWO WAYS TO GET A CLASS LABEL

model.predict(X)

→ straight to True / False, using the built-in 0.5 threshold.

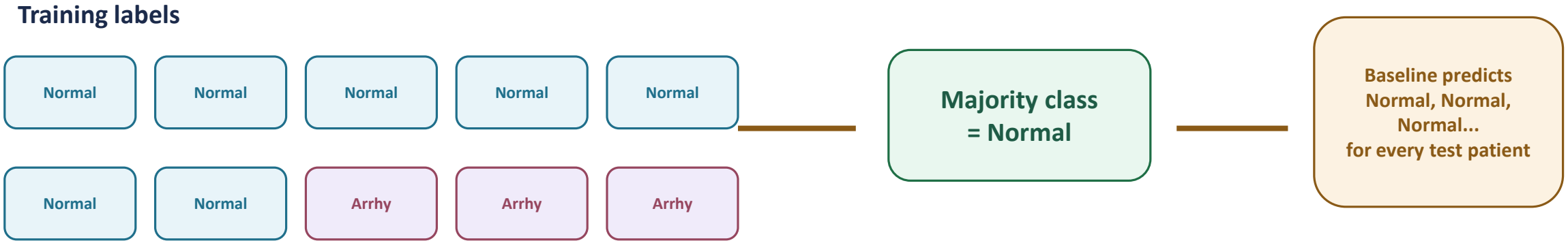
(model.predict_proba(X)[:, 1] > 0.5)

→ the manual version of exactly the same thing — useful if you ever want a different threshold, e.g. 0.3 for a more cautious screening test.

CONCEPT BRIDGE

Baseline accuracy: the lazy model everyone must beat

It predicts the majority class for every single test patient.



Why this matters: If 70% of patients are normal, a model can get 70% accuracy by learning nothing. Logistic regression should clearly beat that lazy baseline.

Baseline is not a serious model — it is a minimum benchmark.

STEP 5 OF 6

Is 72% actually good? Compare against a baseline

```
baseline_guess = y_train.value_counts().idxmax()
baseline_accuracy = (y_test == baseline_guess).mean()

print("Baseline:", round(baseline_accuracy, 2))
# Baseline: 0.58   <- illustrative output
```

LINE BY LINE

1. **`baseline_guess = y_train.value_counts().idxmax()`**
   Finds the single most common class in the training labels — the laziest possible "model."

2. **`baseline_accuracy = (y_test == baseline_guess).mean()`**
   Scores that lazy guess against the real test labels — how often "always guess normal" would be right.

3. **`print("Baseline:", round(baseline_accuracy, 2))`**
   Prints 0.58 — so the real model's 72% is a genuine 14-point improvement, not just exploiting class imbalance.

SIDE BY SIDE

Baseline (always guess majority class)

58%

Logistic Regression (actually learned)

72%

CONCEPT BRIDGE

Confusion matrix: four kinds of outcomes

Rows are actual diagnoses; columns are model predictions.

Predicted

Actual	Normal	Arrhythmia
Normal	True Negative correct normal	False Positive false alarm
Arrhythmia	False Negative missed case	True Positive correct alert

Easy memory trick

Second word = model prediction
Positive = predicted arrhythmia
Negative = predicted normal

First word = was it correct?
True = correct
False = wrong

Clinically: false positive = false alarm; false negative = missed illness.

STEP 6 OF 6

See exactly which mistakes it made

```
cm = confusion_matrix(y_test, predictions)
print(cm)
# [[71, 20], [18, 27]]   <- illustrative output
```

LINE BY LINE

1. **`cm = confusion_matrix(y_test, predictions)`**
   Compares every real label against every predicted label, split into four counts.
2. **`print(cm)`**
   Prints it as a 2×2 grid — rows are actual, columns are predicted.

True negatives (71): normal correctly predicted normal. True positives (27): arrhythmia correctly caught. False positives (20): normal wrongly flagged as arrhythmia. False negatives (18): arrhythmia the model missed — often the most clinically costly mistake.

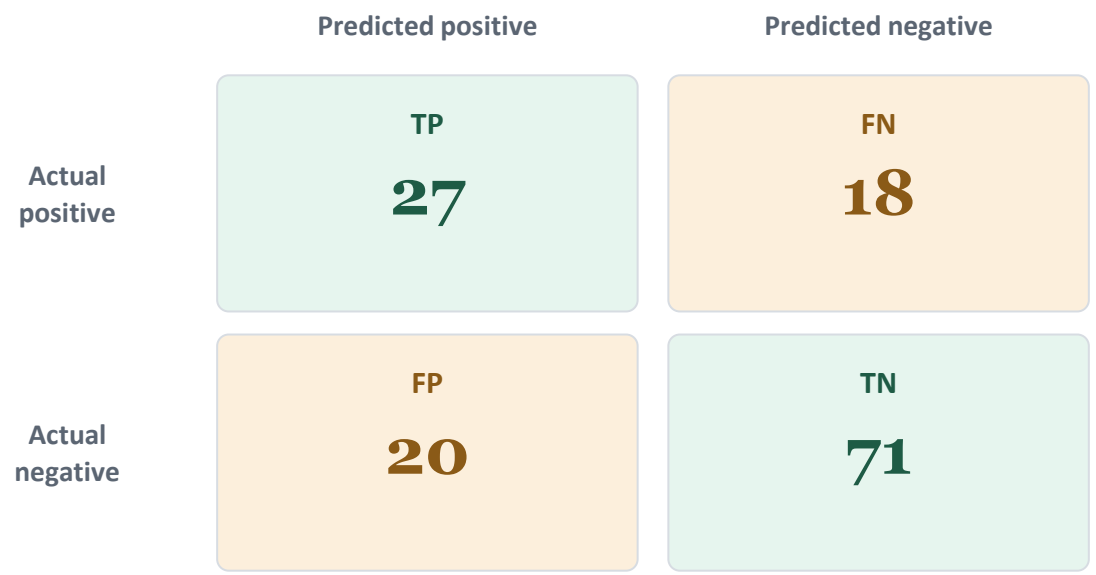
CONFUSION MATRIX

	Pred: normal	Pred: arrhythmia
Actual: normal	71	20
Actual: arrhythmia	18	27

green = correct · orange = mistake

READING A CONFUSION MATRIX VISUALLY

Understanding the four boxes



WHY NOT JUST USE ACCURACY

One accuracy number (72%) hides which kind of mistake the model makes — the four boxes split it apart.

TP (true positive) and TN (true negative) are the two correct outcomes — green.

FP (false positive) and FN (false negative) are the two kinds of mistake — orange, and clinically they are not equally costly.

ONE EXAMPLE PATIENT PER BOX

What each outcome looks like for a real row

True Negative — correctly identified a normal patient

```
Actual: Normal
Prediction: Normal
```

This is a correct result.

False Positive — false alarm

```
Actual: Normal
Prediction: Arrhythmia
```

The model incorrectly gave a positive result. In real life, this patient might undergo unnecessary further testing or experience unnecessary concern.

False Negative — missed arrhythmia

```
Actual: Arrhythmia
Prediction: Normal
```

The model missed a real case. This is often the most clinically costly kind of mistake, since the patient gets no follow-up.

True Positive — correctly caught arrhythmia

```
Actual: Arrhythmia
Prediction: Arrhythmia
```

This is a correct result — the model flagged a real case for review.

PART III

Unsupervised learning

Finding structure in data that has no answer key.

NO LABELS, NO TEACHER

What makes learning “unsupervised”?

- The training data has features only — no correct-answer column to check against
- The model looks purely at similarity and difference between rows
- The goal is to discover structure that wasn't labelled in advance: groups, patterns, or unusual points
- There is no accuracy score, because there is no known right answer — a human still has to interpret what the structure means
- K-means (Lecture 1) is the clustering method already used in this course — we now repeat it on the larger, real arrhythmia data, three steps at a time

ACROSS OTHER DOMAINS

Retail: customers grouped by spending and visit patterns → segments a human then names

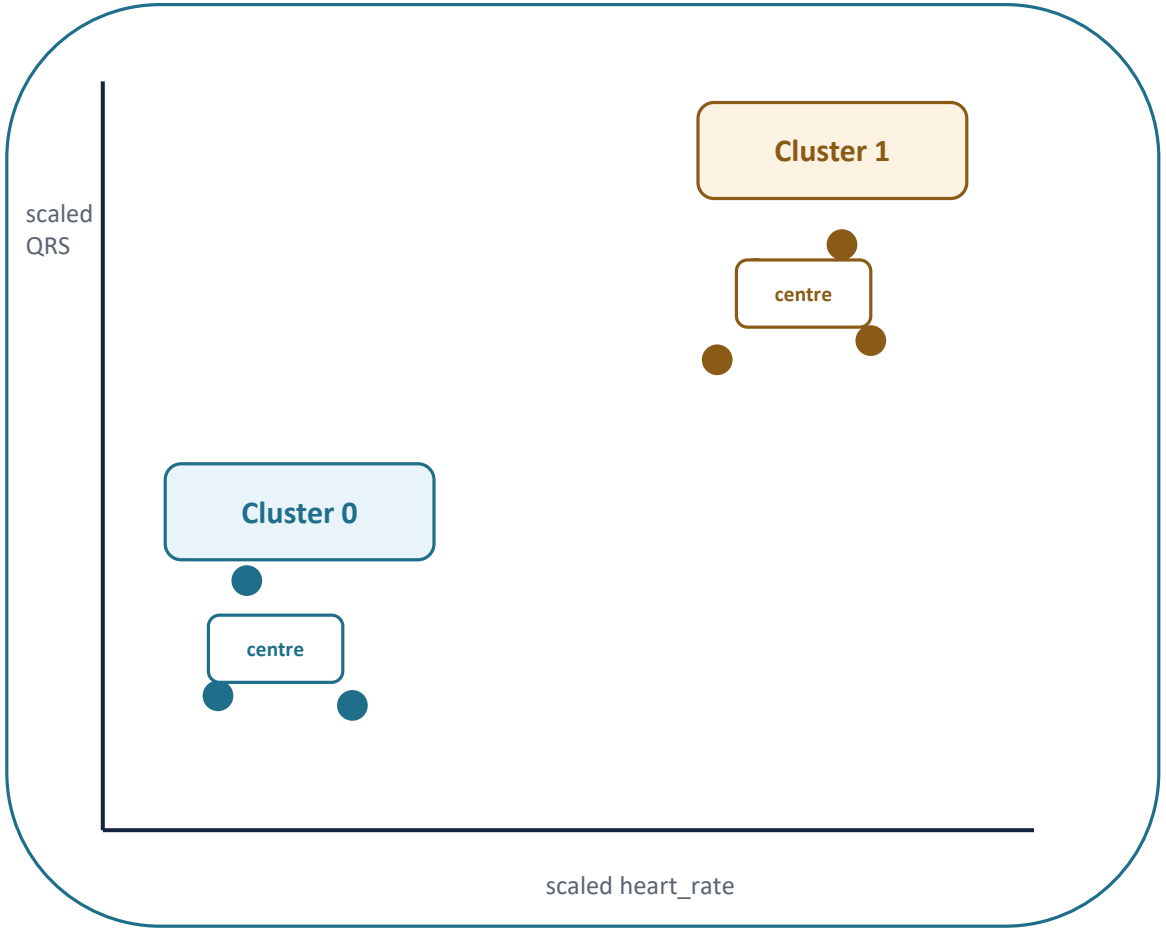
Manufacturing: sensor readings grouped into normal vs unusual machine states

Today: cluster patients purely by ECG measurements, then peek at whether the clusters line up with the real diagnosis — without ever showing the model that label

CONCEPT BRIDGE

K-means groups by similarity, not by diagnosis

It never sees the arrhythmia label while creating clusters.



What K-means knows

- each patient's feature values
- distance between patients
- requested number of clusters

What it does NOT know

- normal vs arrhythmia
- which cluster is "good"
- whether the grouping is clinically meaningful

STEP 1 OF 3

Select features and scale them

```python
from sklearn.preprocessing import StandardScaler

# same df_etl_clean, no relabelling needed
features = ["heart_rate", "qrs_duration", "age"]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_etl_clean[features])

print(X_scaled[:3])   # first 3 rows, illustrative
```

WHY SCALING COMES FIRST

heart_rate, qrs_duration, and age are measured on very different numeric scales — heart_rate might range 60–100, qrs_duration in tens of milliseconds, age in years up to 90+.

Without scaling, KMeans would let whichever feature has the biggest raw numbers dominate its distance calculations — not because it's more important, just because of its units.

StandardScaler rewrites every feature to have a mean of 0 and a standard deviation of 1 — the printed rows should look like small numbers centred around zero.

LINE BY LINE

1. **`from sklearn.preprocessing import StandardScaler`**
   Loads the scaling tool.

2. **`features = ["heart_rate", "qrs_duration", "age"]`**
   Chooses the same three numeric columns used for scaling.

3. **`scaler = StandardScaler()`**
   Creates an empty scaler object.

4. **`X_scaled = scaler.fit_transform(...)`**
   Learns each feature's mean/spread, then rescales every value in one call — a NumPy array comes back.

5. **`print(X_scaled[:3])`**
   Shows the first 3 rows — small numbers centred around zero confirm scaling worked.

CONCEPT BRIDGE

Scaling: fair units before measuring distance

K-means uses distance, so features must be made comparable first.

Feature	Patient A	Patient B	Raw difference	After scaling
Age	70 yrs	50 yrs	20	0.5 SD
QRS duration	80 ms	100 ms	20	2.0 SD

Raw numbers say both gaps are “20”.
That is misleading.

0
average

+1
one SD above

-1
one SD below

±2
unusual

After scaling, every feature is measured as “how unusual is this value compared with that feature’s normal range?”

STEP 2 OF 3

Fit KMeans and assign each patient a cluster

```python
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
df_etl_clean["cluster"] = kmeans.fit_predict(X_scaled)

print(df_etl_clean["cluster"].value_counts())
```

WHAT fit_predict IS DOING

n_clusters=2 mirrors the two real diagnosis groups — chosen deliberately, so the next step can check whether unsupervised clusters recover a pattern the model never saw.

n_init=10 runs the whole clustering process 10 times from different random starting points and keeps the best result — KMeans can otherwise settle on a mediocre grouping by chance.

fit_predict() does both jobs in one call: finds the cluster centres (fit) and immediately returns which cluster each row belongs to (predict) — there's no train/test split, because there's no label to protect from leaking.

LINE BY LINE

1. **`from sklearn.cluster import KMeans`**
   Loads the clustering class.

2. **`kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)`**
   Creates the clusterer — 2 groups, reproducible start, best of 10 attempts kept.

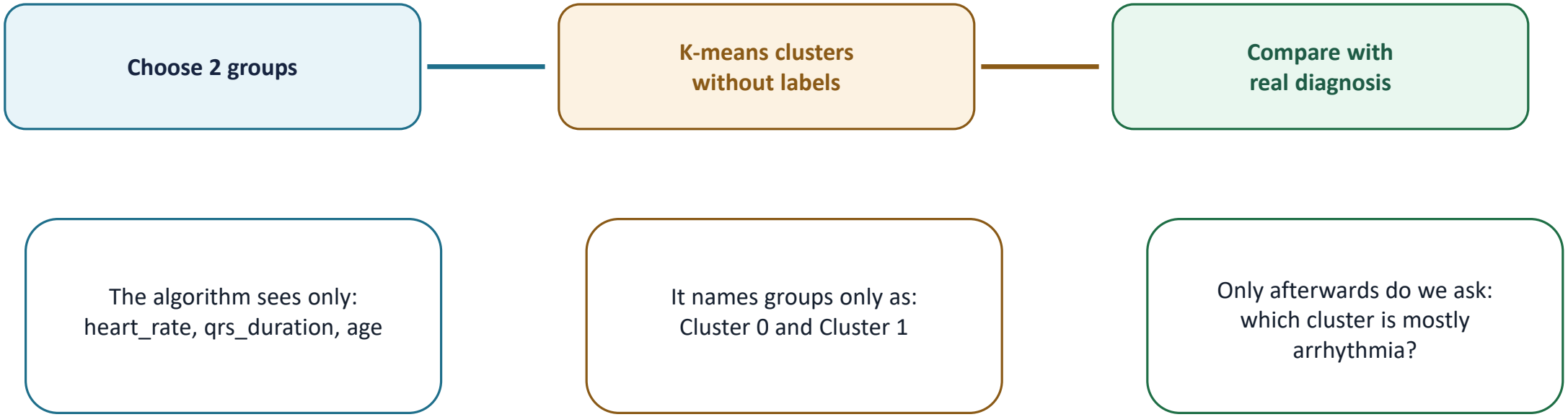
3. **`df_etl_clean["cluster"] = kmeans.fit_predict(X_scaled)`**
   Finds the cluster centres and assigns every row a cluster label, in one call — no train/test split needed.

4. **`print(df_etl_clean["cluster"]...)`**
   Shows how many patients landed in each of the two clusters.

CONCEPT BRIDGE

n_clusters = 2 does not mean “normal vs arrhythmia”

It only means “make two similarity-based groups.”



Better wording: “We choose 2 clusters to see whether the natural groups resemble the two diagnosis categories.”

STEP 3 OF 3

Compare the clusters against the real diagnosis

```
print(pd.crosstab(
    df_etl_clean["cluster"],
    df_etl_clean["arrhythmia_present"]
))
```

LINE BY LINE

1. **`pd.crosstab(`**
   Counts how many rows fall into every combination of two columns.
2. **`df_etl_clean["cluster"], df_etl_clean["arrhythmia_present"]`**
   Compares the unsupervised cluster label against the real diagnosis the model never saw.

Clusters are not automatically "truth." If most of cluster 0 is normal and most of cluster 1 is arrhythmia, that's a strong sign ECG shape alone reflects the diagnosis — an even mix in both would suggest these three features don't separate the groups well.

ILLUSTRATIVE CROSSTAB

	normal	arrhythmia
cluster 0	183	61
cluster 1	64	144

Illustrative only — run it on your data for real counts.

PART IV

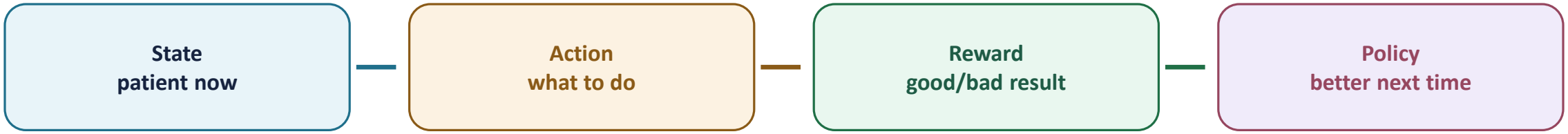
Reinforcement learning

Learning by doing, through trial, error, and reward.

CONCEPT BRIDGE

Reinforcement learning: learning from consequences over time

This is different from a static CSV because actions change what happens next.



Healthcare example

At visit 1, the system chooses: monitor only, request another ECG, or alert a clinician.
The “reward” may come later: correct early detection, fewer false alarms, better outcome.
Over many cases, it learns which action sequence tends to work best.

Classification asks:
“What is this patient?”

Reinforcement learning asks:
“What should we do next?”

That is why there is no code cell for RL in this static dataset.

LEARNING FROM CONSEQUENCES, NOT LABELS

The reinforcement learning loop

1

Agent

The decision-maker being trained — e.g. a treatment-recommendation system.

2

Environment

Everything the agent interacts with and gets feedback from — e.g. the patient and clinical setting.

3

Action

A choice the agent makes at a given moment — e.g. which treatment to try next.

4

Reward

A signal saying how good or bad that action turned out to be — e.g. improved vital signs.

The loop repeats: the agent takes an action, the environment responds with a new situation and a reward, and the agent gradually learns a policy — a strategy for which action tends to earn the best long-run reward. There is no dataset of "correct answers" prepared in advance — the agent has to generate its own experience by acting, which is what separates reinforcement learning from supervised and unsupervised learning, and why there's no code cell for it this week.

WHY IT'S DIFFERENT FROM THE OTHER TWO

Where reinforcement learning shows up

- Game-playing systems that learn strategy purely by playing millions of games against themselves
- Robotics: a robot arm learning to grasp objects through repeated physical trial and error
- Resource allocation: a data-centre system learning to route power and cooling to minimise cost over time
- Healthcare: learning a treatment policy — which action, at which stage, tends to lead to the best patient outcome over a whole course of care
- Recommendation systems that adapt over many interactions, not just one static prediction

NOT COVERED HANDS-ON TODAY

Reinforcement learning needs an environment the agent can repeatedly act in — that's a different setup from a static CSV file, so there is no lab exercise for it this week.

The goal today is conceptual: recognise an RL-shaped problem when you see one, and know how it differs from supervised and unsupervised learning.

PUTTING IT ALL TOGETHER

The three paradigms, side by side

	Supervised	Unsupervised	Reinforcement
Data needed	Labelled examples	Unlabelled examples	An environment to act in
Goal	Predict a known label	Discover structure	Learn a good policy
Feedback signal	Correct answer per row	None — similarity only	Reward, often delayed
Today's code	LinearRegression / LogisticRegression	KMeans + StandardScaler	Conceptual only